ETA Prediction & Live Location Tracking

ETA Prediction & Live Location Tracking for Quick Grocery Delivery

A system design framework for last-mile delivery estimation and real-time tracking

---

Table of Contents

1. Reframing the Problem
2. ETA Prediction: How It Actually Works
3. Live Location Tracking: How the Moving Dot Gets There
4. Data You Need
5. Questions to Ask If This Data Doesn't Exist Yet
6. Factors You Didn't Mention — But That Matter
7. Constraints That Shape This System
8. Base Assumptions

---

1. Reframing the Problem

You're actually looking at two coupled systems, not one:

1. **ETA Prediction** — an estimation/regression problem: given a store, a customer address, current conditions, and a rider, predict time-to-doorstep.
2. **Live Location Tracking** — a real-time streaming/infra problem: continuously capture, clean, and broadcast a moving GPS point to a customer's phone with low latency.

They're coupled because ETA isn't a one-time calculation — it needs to **re-estimate continuously** as the rider actually moves, using the same location stream that powers the live map. Get the architecture right and both problems share most of their plumbing.

---

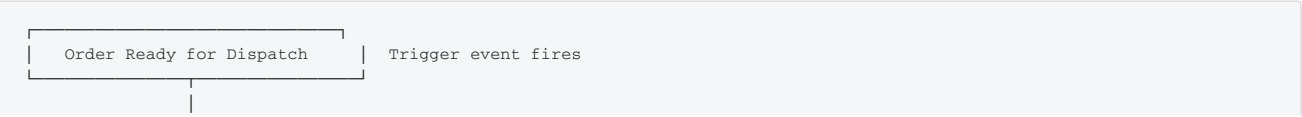
2. ETA Prediction: How It Actually Works

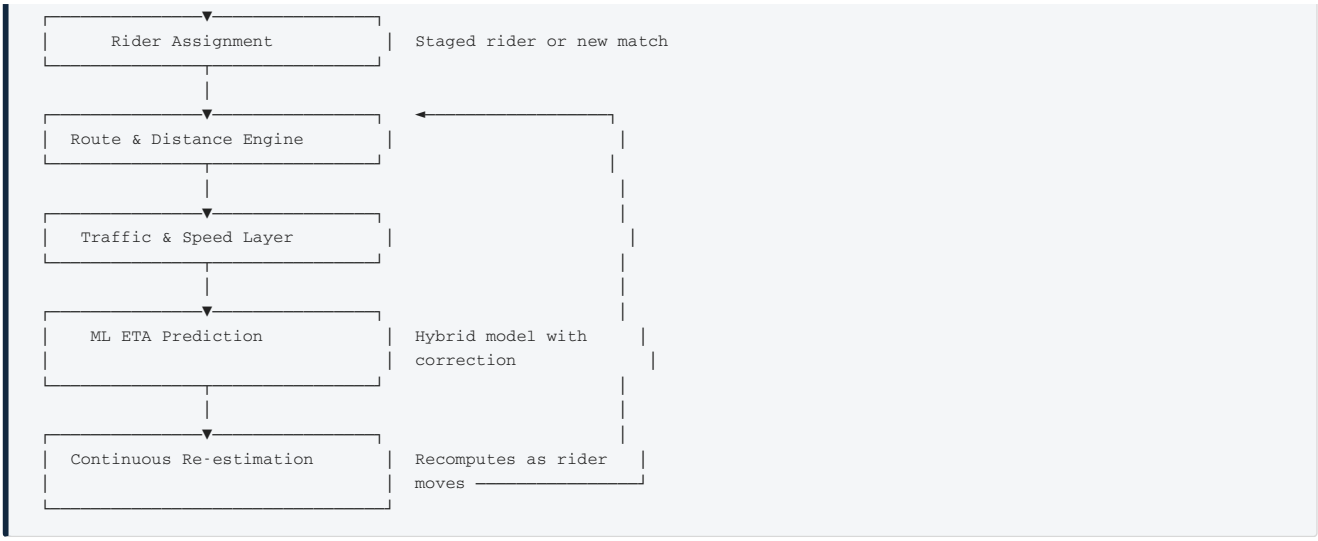
2.1 The Two Components of ETA at "Packed and Ready to Dispatch"

Component	What It Captures	Typical Magnitude
Rider-to-store time	If no rider is already staged at/near the store, time for one to become available and reach it	0 min (if pre-staged) to 5–10 min
Store-to-customer time	The actual delivery leg — this is what most systems mean by "ETA"	Bulk of the estimate
Buffer/handover time	Parking, building access, elevator wait, OTP/handoff confirmation	1–3 min, often underestimated

Most naive systems only model the middle component (straight-line or road-network distance ÷ average speed) — but the first and third are where real systems usually get embarrassed by wildly wrong ETAs.

2.2 The Computation Pipeline





Each stage builds on the previous one, and the last stage loops back into the route engine — as new GPS pings arrive, the system re-queries "distance/time from wherever the rider actually is now" rather than sticking with the original estimate.

2.3 The Four Layers of the Estimate

Layer 1 — Routing engine (deterministic baseline). A road-network graph (nodes = intersections, edges = road segments with length + speed limit + one-way rules) run through a shortest-path algorithm — Dijkstra/A* for on-demand queries, or precomputed contraction hierarchies for speed at scale. This is what OSRM, Valhalla, Google Directions API, or Mapbox give you: a route polyline + a "free-flow" time estimate. On its own this is *always* wrong in real conditions because it ignores current congestion.

Layer 2 — Real-time traffic/speed layer. Overlay live conditions onto that graph, two ways to get this: - **Buy it** — Google Maps/HERE/Mapbox traffic APIs (fast to integrate, recurring cost, black-box) - **Build it** — aggregate your own fleet's GPS probe data: bucket road segments (geohash/S2 cells), compute rolling average actual speed per segment per time-of-day, and feed that back as edge weights. This is what most large delivery companies eventually do, since it's free once you have GPS volume and captures *your* riders' actual behavior.

Layer 3 — ML correction model (the part that makes it good). Routing-engine time is a *feature*, not the final answer. Train a model — gradient-boosted trees (LightGBM/XGBoost) to start, graph neural networks over the road network if you have serious scale (roughly what Google Maps and Uber's DeepETA do) — on historical **actual** delivery times as the label, with features like:

- Route-engine baseline time & distance
- Time of day, day of week, holiday/festival flag
- Weather (rain especially — huge effect on two-wheeler speed)
- Rider type (bike/scooter/bicycle/on-foot) and that rider's historical average speed
- Local area density (dense mixed-traffic markets vs. wide arterial roads)
- Number of orders batched with this one
- Building type at destination (apartment complex/gated society vs. standalone house) — proxy for handover time
- Historical delivery time for this exact store→locality pair (a strong prior)

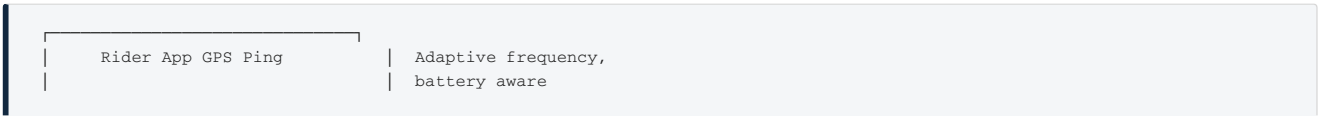
The model predicts a **residual correction** on top of the routing engine's baseline, or predicts total time directly with the baseline as one input feature. Either works; residual-correction is usually more stable and interpretable.

Layer 4 — Uncertainty, not a point estimate. Don't just predict a number — predict a **range** using quantile regression (predict P50 and P90). Show the customer "10–14 min" rather than a false-precision single number, and widen the interval automatically when confidence is low (new rider, unfamiliar area, heavy rain).

Layer 5 — Continuous re-estimation. Every time a new GPS ping arrives (or every 15–30 seconds), recompute remaining distance from the rider's *current* snapped position and re-run through the same model. This is why the tracking pipeline and the ETA pipeline share infrastructure.

---

3. Live Location Tracking: How the Moving Dot Gets There





Rider app → ingestion. The rider's phone sends GPS pings — typically every 3–5 seconds while moving, throttled to every 15–30 seconds when stationary or far from the destination (adaptive frequency to save battery). These go over HTTPS or a persistent socket to an ingestion layer built for high write throughput (Kafka or a managed pub/sub), since this happens for every active rider simultaneously across the city.

Map matching — the unglamorous but essential step. Raw phone GPS is noisy — 5–20 meter error is normal, worse in dense urban areas ("urban canyon" effect from tall buildings), and a raw point will often show a rider "driving through buildings" or on the wrong side of a divided road. A **map-matching algorithm** (Hidden Markov Model based — the rider's true position is a hidden state, GPS pings are noisy observations, and you infer the most likely path along actual road segments; this is what OSRM's map-matching module and most fleet-tracking systems use) snaps the raw point onto the road network. A simpler Kalman filter can also smooth jitter if the full HMM machinery isn't needed initially.

Real-time store. The snapped position is written to a fast key-value store (Redis with geospatial commands, or DynamoDB/Cassandra) keyed by order_id/rider_id — this needs sub-100ms read/write since it's on the hot path for both the customer's map and the ETA re-estimation.

Push to customer. A WebSocket (or MQTT) gateway maintains a live channel per active order; when a new position lands, it's pushed immediately rather than the customer app polling. Polling every 5–10s is a reasonable fallback if WebSocket infra isn't built yet, but it costs more battery/data on the customer side and feels laggier.

Client-side interpolation. Since real pings only arrive every few seconds, the app doesn't just jump the marker — it **animates/interpolates** the marker smoothly between the last two known points along the road-snapped path, so movement looks continuous rather than teleporting.

Degraded-mode handling. When GPS is lost (tunnel, dead zone, app backgrounded and killed by OS battery optimization — very common on Android), fall back to dead-reckoning (extrapolate from last known speed/heading for a short window) and simultaneously widen the ETA confidence interval, rather than showing a frozen or wrong marker indefinitely.

---

4. Data You Need

Category	Specifics
Historical trip data	GPS trace (lat/long every few seconds) per trip, tagged to order_id, with actual pickup and delivery timestamps — this is your ETA model's ground truth
Road network	Graph with segment length, speed limits, one-way/turn restrictions — either self-hosted (OSM-based) or licensed
Historical + real-time traffic	Either third-party API, or derived from your own fleet's aggregated speeds per road segment per time-of-day
Rider data	Vehicle type, historical average speed, tenure/experience, current status (idle/en route/multiple pickups)

Order metadata	Item count/weight, COD vs. prepaid, number of orders batched together
Address data	Customer's dropped pin accuracy, building type (individual house vs. high-rise apartment), delivery instructions
Weather	Real-time + historical, especially rain — outsized effect on two-wheeler speed
Events/closures	Local road closures, festivals, processions, construction — often missing from generic map data in Indian cities

---

5. Questions to Ask If This Data Doesn't Exist Yet

- **"Do we log the full GPS trace per trip, or only start/end points?"** If only start/end, you can't train a good ETA model or debug tracking issues — start logging full traces immediately; this is the single highest-leverage fix.
- **"Are we buying a routing/traffic API (Google, Mapbox, HERE) or self-hosting (OSRM/Valhalla on OSM data)?"** This is a build-vs-buy call with real cost implications at scale — worth revisiting even if already decided, since call volume in quick commerce (recalculating ETA every 15–30s per active order) can make third-party API costs balloon fast.
- **"What's the current GPS ping frequency, and is it adaptive?"** If it's fixed-frequency regardless of rider speed/distance, there's low-hanging fruit for both battery savings and tracking smoothness.
- **"Do we already have a WebSocket/pub-sub gateway, or is the app currently polling?"** Determines how much new infra this needs.
- **"How reliable is network connectivity in our served localities?"** In tier-2/3 Indian cities or dense urban interiors, 3G/patchy 4G is still common — affects whether offline buffering on the rider app is needed.
- **"Is address geocoding (the customer's dropped pin) audited for accuracy?"** A surprising amount of "wrong ETA" complaints are actually "wrong destination pin," not a bad ETA model.

---

6. Factors You Didn't Mention — But That Matter

- **Order batching / multi-stop routing.** If a rider carries 2–3 orders in one trip (common in quick commerce for density), ETA for each order depends on the **sequence** the rider delivers them in — a mini traveling-salesman optimization the dispatch system needs to solve, which changes every individual order's ETA the moment a new order gets added to that rider's route.
- **Rider assignment/dispatch algorithm.** ETA doesn't start with "rider already at store" — it starts with whether a rider is even assigned yet. The dispatch algorithm (nearest-available-rider matching, often itself a small optimization problem balancing rider idle time against customer wait) directly feeds the first component of ETA.
- **Handover/access friction.** Elevator wait in a high-rise, security gate sign-in at a gated society, finding parking — often 1–3 minutes that pure routing math never accounts for, but very predictable per-address once delivery history exists at that address.
- **Payment collection time.** Cash-on-delivery orders take measurably longer at the door (change handling) than prepaid — small but real, and systematically biases ETA if ignored.
- **Vehicle-specific routing rules.** A bike/scooter can take routes and shortcuts a car cannot; routing profile needs to differ by vehicle type.
- **Map/geocoding quality in the local market.** In many Indian cities, informal roads, unmapped shortcuts, and imprecise customer-dropped pins are a bigger source of ETA error than the algorithm itself — a data-quality problem, not a modeling problem.
- **Customer availability.** If the customer isn't home/reachable, that's a delay the system should communicate rather than just silently blowing the ETA.
- **Weather-driven route/speed changes**, especially monsoon flooding of low-lying routes in cities like Mumbai — a road that's fastest on a dry day may be impassable after rain, which a static routing graph won't know unless fed real-time road-condition signals.
- **Perishable/cold-chain constraint.** For frozen or ice-cream SKUs, the system may need to bias toward the fastest route even if slightly costlier — a business rule layered on top of pure ETA-minimization.
- **Confidence communication, not just prediction.** Showing a shrinking/expanding range (and explaining "traffic is heavier than usual" type context) builds trust better than a single number that's occasionally very wrong.

---

7. Constraints That Shape This System

Constraint Type	Specifics
Latency	Location updates should reach the customer app within ~1–5 seconds end-to-end; ETA recompute cycle every 15–30s is usually sufficient for grocery (less demanding than ride-hailing's sub-second bar)
Scale	Needs to handle potentially tens of thousands of concurrent active deliveries city-wide with continuous read/write — push architecture must be horizontally shardable, typically by geography
Battery & data usage	Frequent GPS pings drain rider battery and use mobile data — adaptive frequency (faster near destination, slower when idle) is a real design trade-off
Network reliability	Patchy connectivity requires offline buffering on the rider app with sync-on-reconnect, and graceful degraded states on the customer app
Cost	Third-party map/traffic API costs scale with call volume — recalculating ETA every 15–30s per order can get expensive; self-hosted routing becomes attractive past a certain scale
Privacy/compliance	Location sharing should be scoped to the active delivery window only, purged after delivery, and compliant with local data protection law (India's DPDP Act)
Map data quality	Especially in developing markets — informal roads, incorrect one-ways, imprecise customer pins all directly degrade both routing and ETA accuracy regardless of model sophistication
Explainability/trust	A customer-facing ETA that's wildly wrong even occasionally damages trust more than a slightly-wider-but-reliable range — favors quantile/confidence-interval approaches over point estimates
Rider app reliability	OS-level background-process killing (especially Android battery optimization) can silently stop location updates — needs foreground-service handling and fallback detection

8. Base Assumptions

1. You already have (or can quickly get) a GPS-instrumented rider fleet — the app already captures location, just maybe not optimally.
2. There's a routing engine or map API in use today, even if basic (Google Directions, Mapbox, or self-hosted OSRM).
3. Delivery is single-city/dense-urban with two-wheelers as the dominant vehicle type (typical for Indian quick commerce) — this shapes routing profile assumptions.
4. Real-time customer-facing latency tolerance is in the few-seconds range, not sub-second (unlike ride-hailing).
5. Orders may be batched (multiple orders per rider trip) — if this isn't true for your model, the dispatch/sequencing complexity drops significantly.

Document prepared as a system design framework for ETA prediction and real-time delivery tracking in quick-commerce grocery delivery.